Тема 2. Променливи. Стандартни типове. Изрази. Преобразуване на типа. Операции.

Съдържание

- Как компютрите представят данните
- Променливи
- Идентификатори
- Типове данни
- Инициализация на променливи
- Определяне на типа
- Вход-изход в C++
- Операции - аритметични, логически, побитови

Представяне на данните

- Битове (Bits)
  - *Bit – "Binary digIT"*
  - *Двоичната цифра може да съдържа 0 или 1*
  - *1 и 0 съответстват на включване и изключване, съответно*
- Байт (Bytes)
  - *Комбинация от 8 бита*
  - *Представлява един знак, като например буквата A*
  - *За представяне на данни компютрите използват числова система с основа 2 или двоична бройна система*

	10^3 (1000)	10^2 (100)	10^1 (10)	10^0 (1)	
	1	3	2	6	
	1 times 10^3 (1000)		=	1000	
+	3 times 10^2 (100)		=	300	
+	2 times 10^1 (10)		=	20	
+	6 times 10^0 (1)		=	6	
				1326	

Позиционна нотация с основа 10 на 1326

Представяне на данните

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
(128)	(64)	(32)	(16)	(8)	(4)	(2)	(1)
0	**1**	**1**	**0**	**1**	**0**	**0**	**1**

	0 times 2^7 (128)	=	0
+	1 times 2^6 (64)	=	64
+	1 times 2^5 (32)	=	32
+	0 times 2^4 (16)	=	0
+	1 times 2^3 (8)	=	8
+	0 times 2^2 (4)	=	0
+	0 times 2^1 (2)	=	0
+	1 times 2^0 (1)	=	1
			105

Десетичен еквивалент на 01101001

Представяне на данните

Decimal value	Binary equivalent
0	00000000
1	00000001
2	00000010
3	00000011
4	00000100
5	00000101
6	00000110
7	00000111
8	00001000
...	...
254	11111110
255	11111111

Двоичен еквивалент на избрани десетични стойности

Представяне на данните

- Множество от символи
- С 8 бита може да се представят 2^8 или 256 различни десетични стойности, вариращи от 0 до 255 (256 различни знака)
- Unicode – Множеството от символи
  - *Използва 16 бита, за представя символи (могат да бъдат представени 2^{16}, или 65 536 уникални символи)*
- American Standard Code for Information Interchange (ASCII) – подмножество на Unicode (Първите 128 символа са същите)

Storage capacity	Size in bytes	Abbreviation
Kilobyte	2^{10} (1,024)	KB
Megabyte	2^{20} (1,048,576)	MB
Gigabyte	2^{30} (1,073,741,824)	GB
Terabyte	2^{40} (1,099,511,627,776)	TB
Petabyte	2^{50} (1,125,899,906,842,624)	PB
Exabyte	2^{60} (1,152,921,504,606,846,976)	EB
Zettabyte	2^{70} (1,180,591,620,717,411,303,424)	ZB
Yottabyte	2^{80} (1,208,925,819,614,629,174,706,176)	YB

Общи абревиатури за представяне на данни

Променливи

- Нека си представим, че трябва да запаметим числото 5, след което едновременно с него да запаметим 2.
  - *Запаметихте в паметта си две различни стойности (5 и 2).*
  - *Прибавите 1 към първата стойност. Сега трябва да помните числата 6 (от 5+1) и 2 в паметта си.*
  - *Извадим стойностите една от друга, резултатът е 4.*
- Процесът описан по-горе е само една малка извадка от това какво могат да извършват компютрите с две променливи.
- За да се извършат споменатите операции в C++, то трябва да се използват следните команди:

```
a = 5;
b = 2;
a = a + 1;
result = a - b;
```

Идентификатори

- Името на променливата се нарича идентификатор.
- ***Валиден идентификатор*** е последователност от една или повече латински букви, цифри, или знакът долна черта (_).
  - ***!!! Интервали, препинателни знаци или символи не могат да са част от името на един идентификатор.***
  - *Идентификаторите винаги започват с буква.*
  - *Могат да започват с долна черта – в повечето случаи тези идентификатори се считат за резервирани за специфични за компилатора ключови думи или външни идентификатори, както и идентификатори с две последователни долни черти, където и да е в името.*
  - *В никакъв случай идентификаторите не могат да започват с цифра.*

Идентификатори

- Pascal case
  - *Първата буква на всяка дума е главна буква*
  - *Класове, методи, имена на пространства и свойства*

- Camel case
  - *Унгарска нотация*
  - *Първата буква на първата дума на идентификатора е малка, а всяка следваща дума започва с главна буква*
  - *Променливи и обекти*

- Uppercase
  - *Всички думи са с главни букви*
  - *Константи*

Идентификатори

- ***!!! Езикът C++ е "case sensitive".***
  - *Това означава, че променлива написана с главни букви не е еквивалентна на друга написана със същото име, но с малки букви.*
- Пример:
  - *Променливата* **RESULT** *не е същата като променливата* **result** *или променливата* **Result**.
  - *Това са три различни идентификатора, които съответстват на три различни променливи.*

Valid identifiers		
studentName	age	numberOfCourses
soc_sec_number	departureTime	course1
AmountOwed	count_of_Values	taxAmount
n	streetAddress	zipCode
roomSize	courseName	x3
moreData	bookTitle	homeRuns
pointsScored	CLUB_NAME	exam4

Invalid identifiers	Description of violation
soc sec number	Embedded space
int	Reserved keyword
3rdDay	Begins with a digit
room #	Special symbol other than underscore and an embedded space
first-name	Special symbol other than underscore
A number	Embedded space
class	Reserved keyword

Служебни думи

- C++ използва множество от ключови думи, за да идентифицира различни операции и описание на данните, следователно идентификаторите създадени от програмистите не могат да съвпадат с тези ключови думи.

- Списък със стандартни ключови думи в C++:

alignas, alignof, and, and_eq, asm, auto, bitand, bitor, bool, break, case, catch, char, char16_t, char32_t, class, compl, const, constexpr, const_cast, continue, decltype, default, delete, do, double, dynamic_cast, else, enum, explicit, export, extern, false, float, for, friend, goto, if, inline, int, long, mutable, namespace, new, noexcept, not, not_eq, nullptr, operator, or, or_eq, private, protected, public, register, reinterpret_cast, return, short, signed, sizeof, static, static_assert, static_cast, struct, switch, template, this, thread_local, throw, true, try, typedef, typeid, typename, union, unsigned, using, virtual, void, volatile, wchar_t, while, xor, xor_eq

- Специфичните компилатори, могат да поддържат и допълнително множество от ключови думи.

Основни типове данни

- Стойностите на променливите се съхраняват някъде неизвестно къде в паметта на компютъра под формата на последователност от 0 и 1.
- За нашата програма не е нужно да се знае точното местоположение на съхранената променлива, просто се обръщаме към нея по името ѝ.
- Това, което е изключително важно за нашата програма е типа на данните, които се съхраняват или ще се съхраняват в променливата.
  - *Не е едно и също дали ще съхраняваме просто едно цяло число или буква или много голямо число с плаваща запетая; въпреки, че при вътрешното си представяне всички видове стойности се съхраняват под формата на 1 и 0.*
- Последователностите от 1 и 0 не се интерпретират по един и същи начин, а различните типове данни се нуждаят и от различно количество памет, в което да разположат стойностите.

Основни типове данни

- Основните типове данни са базови типове, които са вградени в езика и чрез които се изразяват основните възможности за съхранение на данни, поддържани стандартно от повечето системи.
- Основно се делят на:
  - ***Символни типове:*** *могат да представят единични символи като* `'A'` *или* `'$'`*. Най-базовият тип е* `char`*, който е едно-байтов символ. Съществуват и други типове за по-обширно множество от символи.*
  - ***Целочислени типове:*** *използват се за съхранение на цели числа, като например* `7` *или* `1024`*. Съществуват в различни размери и могат да бъдат със знак или без знак, в зависимост от това дали е необходимо да се съхраняват отрицателни стойности.*
  - ***Типове с плаваща запетая:*** *използват се за съхраняване на реални стойности, като* `3.14` *или* `0.01`*, с различно ниво на точност.*
  - ***Булеви типове:*** *булевият тип в C++ е известен като* `bool`*, може да съхранява само две състояния -* `true` *или* `false`*.*

Основни типове данни в C++

Група	Име на типа	Размер/точност
Символни типове	**char**	Точно един байт по размер. Поне 8 бита.
	char16_t	Не по-малък от char. Поне 16 бита.
	char32_t	Не по-малък от char16_t. Поне 32 бита.
	wchar_t	Може да се използва за най-големия символен тип, който се поддържа стандартно.
Целочислени типове (със знак)	**signed char**	Също като char. Поне 8 бита.
	signed **short** *int*	Не по-малък от char. Поне 16 бита.
	signed **int**	Не по-малък от short. Поне 16 бита.
	signed **long** *int*	Не по-малък от int. Поне 32 бита.
	signed **long long** *int*	Не по-малък от long. Поне 64 бита.
Целочислени типове (без знак)	**unsigned char**	Същите като техните съответствия със знак.
	unsigned short *int*	
	unsigned *int*	
	unsigned long *int*	
	unsigned long long *int*	

Основни типове данни в C++

Група	Име на типа	Размер/точност
Типове с плаваща запетая	`float`	Поне 4 байта
	`double`	Точност не по-малка от на float. Поне 8 байта
	`long double`	Точност не по-малка от double
Булев тип	`bool`	
Void тип	`void`	Не заема памет
Null указател	`decltype(nullptr)`	

- Имената на определени целочислени типове могат да се записват съкратено, като се изпускат частите, които се са с шрифт *Italic*.
  - *Напр.,* ***signed short int*** *може да се съкрати като* ***signed short****,* ***short int****, или просто* ***short****. Всички представляват един и същ основен тип.*
- В рамките на всяка група от таблицата, единствената разлика е размера, който заемат в паметта.
- Първият тип в групата винаги е най-малкият, а последния най-големият.

Основни типове данни в C++

- Обърнете внимание, че освен за типа *char* (който заема точно един байт), за никой от основните типове не е посочен стандартен размер, а само минимален размер.
- В много случаи размера не е точно минимален размер.
- Това не означава, че тези типове са с неясен размер, а просто няма стандартен размер за всички компилатори и машини; всяка реализация на компилатор може да специфицира собствени размери на тези типове, които ще съответстват по най-подходящ начин на архитектурата, на която ще се изпълнява програмата.
- Тази доста основна спецификация на типа дава на езика C++ доста гъвкавост, за да се адаптира да работи програмата оптимално на всички видове платформи, както настоящи така и бъдещи.

Основни типове данни в C++

- Дадените типове данни се представят с битове.
- **Колкото повече битове има един тип, толкова по-точни и повече стойности, той може да представя, но също така консумира повече памет:**

Размер	Уникални стойности, които могат да се представят	Бележки
8-bit	256	$= 2^8$
16-bit	65536	$= 2^{16}$
32-bit	4294967296	$= 2^{32}$ (~4 billion)
64-bit	18 446 744 073 709 551 616	$= 2^{64}$ (~18 billion billion)

Основни типове данни в C++

- За целочислените типове да имат повече представени стойности, означава че диапазона от стойности, които могат да се представят е по-голям.
  - *Пример, едно 16-битово без знаково цяло число може да представи 65536 различни стойности в диапазон от 0 до 65535, докато съответният му знаков тип в повечето случаи ще може да представи стойности от -32768 до 32767.*

- Обърнете внимание, че **диапазона на типа на положителните стойности е приблизително разполовен при типа на стойности със знак**, на база на факта че един от 16-те бита се използва за знак.
  - *Това е сравнително скромна разлика в диапазона и рядко оправдава използването на беззнакови типове, съдържащи единствено диапазон от положителни стойности, които могат да представляват.*

Основни типове данни в C++

- За типовете с плаваща запетая, типът определя тяхната точност, като когато има повече или по-малко битове за техните значещи цифри и експонента.
- Ако размерът и точността не са от значение, то тогава стандартният избор е *char*, *int*, и *double*, за да се представят съответно символи, цели числа и числа с плаваща запетая.
  - *Останалите типове от техните групи се ползват в много специфични случаи.*
- Свойствата на фундаменталните типове в конкретна система и в имплементацията на компилатора могат да бъдат проверени, чрез използване на класовете *numeric_limits* (вижте стандартния заглавен файл <limits>).
- Ако по някаква причина са необходими типове с конкретни размери, в библиотеката <cstdint> са дефинирани някои псевдоними с фиксиран размер.

Основни типове данни в C++

- Описаните по-горе типове (*char*, *int*, *float* и *bool*) са известни като аритметични типове.
- Съществуват още два основни типа:
  - *void, който идентифицира липсата на тип;*
  - *nullptr, който е специален тип за указател.*
- C++ поддържа голямо разнообразие от типове, базирани на основните типове, обсъдени до тук; тези други типове са известни като *комбинирани/съставни типове данни* и са едно от основните предимства на езика C++.

Декларация на променливи

- C++ е строго типизиран език и изисква всяка променлива да бъде декларирана с нейния тип преди да бъде използвана за пръв път.
  - *Помага на компилатора да определи размера памет, който да задели и как да интерпретира нейната стойност.*
- Синтаксис за деклариране на нова променлива в C++:

  тип име;

  - *Първо се посочва типа, а след това се създава идентификатор или име на променливата.*
- Пример:

  ```
  int a;
  float myNumber;
  ```

  - *Първият израз декларира променлива от тип* ***int*** *с идентификатор* ***a****.*
  - *Вторият израз декларира променлива от тип* ***float*** *с идентификатор* ***myNumber****.*
- Веднъж декларирани, променливите ***a*** и ***myNumber*** могат да се използват в рамките на тяхната област на видимост.

Декларация на променливи

- Ако се налага декларирането на повече от една променлива от един и същи тип, то те всички могат да бъдат декларирани заедно в един израз, като идентификаторите им се разделят със запетая.
- Пример:

```
int a, b, c;
```

  - *Чрез горния израз са декларирани три променливи (a, b и c), всички с тип int.*
  - *Горният израз има абсолютно същото значение като:*

```
int a;
int b;
int c;
```

Декларация на променливи

- Нека разгледаме една цяла C++ програма, чийто код представлява съхранението на стойностите, с които започнахме лекцията:

```
// operating with variables
#include <iostream>
using namespace std;

int main ()
{
  // declaring variables:
  int a, b;
  int result;

  // process:
  a = 5;
  b = 2;
  a = a + 1;
  result = a - b;

  // print out the result:
  cout << result;

  // terminate the program:
  return 0;
}
```

Изход:

```
4
```

Инициализация на променливи

- Когато се декларират променливите в предходния пример, те имат неопределена стойност, докато за пръв път им бъде присвоена стойност.
- Възможно е да се зададе първоначална стойност на дадена променлива още при нейната декларация. Това се нарича ***инициализация*** на променливата.
- В C++ има три начина за инициализация на променливи.
  - *Всички те са еквивалентни по между си и показват еволюцията на езика през годините.*
  1. *Първият вид е известна като **C-стил на инициализация** (тъй като се наследява от езика C). Състои се от задаването на знака за равно, следван от стойността, с която ще се инициализира променливата:*

     тип идентификатор = начална_стойост;
  - *Например, за да се декларира една променлива от тип **int** наречена **x** и да я инициализираме със стойност **0**, то трябва да напишем следния израз:*

    ```
    int x = 0;
    ```

Инициализация на променливи

2. *Вторият метод известен като **конструкторна инициализация** (въведен в езика C++), загражда началната стойност в скоби (()):*

```
type identifier (initial_value);
```

- *Пример:* `int x (0);`

3. *Последният трети метод за инициализация, известен като **единна инициализация**, е подобна на втория метод, но се използват кадрави скоби ({ }) (това е въведено при актуализацията на стандарта C++ през 2011):*

```
type identifier = {initial_value};
```

- *Пример:* `int x = {0};`

Инициализация на променливи

Пример:

```
#include <iostream>
using namespace std;

int main ()
{
    int a=5;    // initial value: 5
    int b(3);   // initial value: 3
    int c={2};  // initial value: 2
    int result; // initial value undetermined

  a = a + b;
  result = a - c;
  cout << result;

  return 0;
}
```

Изход:

```
6
```

Определяне на типа: auto и decltype

- Когато се инициализира нова променлива, **компилаторът може да определи типа на променливата автоматично при инициализацията**.
  - *Трябва да се използва **auto** като спецификатор на тип за променливата:*

```
int foo = 0;
auto bar = foo;  // същото като: int bar = foo;
```

  - *bar е декларирана с тип **auto**, следователно типът на bar е типът на стойността използвана за инициализацията ѝ (използва се типът на foo, който е **int**).*

- Променливи, които не са инициализирани могат да се възползват от автоматичното определяне на типа със спецификатора *decltype*:

```
int foo = 0;
decltype(foo) bar;  // същото като: int bar;
```

  - *bar е декларирана да има същия тип като foo.*

Определяне на типа: auto и decltype

- *auto* и *decltype* са много мощни функции добавени.
  - *Но характеристиките за приспадане на типа, които въвеждат, са предназначени да бъдат използвани или когато типът не може да бъде получен по друг начин, или когато се използват, за да подобрят четливостта на кода.*
- Двата примера по-горе не бяха нито едно от двете:
  - *Типът на* `foo` *е ясно видим, следователно директно може да се напише* `int`*.*
  - *В посочените примери водят до намаляване на четливостта, тъй като при четенето на кода трябва да се търси типът на* `foo`*, за да се знае истинският тип* `bar`*.*

Операции

- Оператор за присвояване (=)
- Аритметични операции (+, -, *, /, %)
- Комбинирано присвояване (+=, -=, *=, /=, %=, >>=, <<=, &=, ^=, |=)
- Инкремент и декремент (++, --)
- Релационни операции и операции за сравнение (==, !=, >, <, >=, <=)
- Логически операции (!, &&, ||)
- Условен троен оператор (?)
- Оператор запетая (,)
- Побитови оператори (&, |, ^, ~, <<, >>)
- Оператор за явно преобразуване на типа
- *sizeof*

Оператор за присвояване (=)

- Операторът за присвояване се използва за задаване на стойност на променлива.

```
x = 5;
```

  - *Този израз присвоява целочислената стойност 5 на променливата x.*

- Операторът за присвояване винаги се изпълнява от дясно на ляво и никога на обратно:

```
x = y;
```

  - *В този израз на променливата x се присвоява, стойността съдържаща се в y.*
  - *В момента на изпълнение на израза, старата стойност на x се загубва и се замества от стойността съхранена в променливата y.*
  - *Присвояваме текущата стойност на y в x.*
  - *Следователно, ако по-късно y промени стойността си, това няма да се отрази на стойността съхранена в x.*

Оператор за присвояване (=)

Пример:

```
#include <iostream>
using namespace std;

int main() {
    int a, b;          // a:?,  b:?
    a = 10;            // a:?,  b:?
    b = 4;             // a:?,  b:?
    a = b;             // a:?,  b:?
    b = 7;             // a:?,  b:?

    cout << "a:";
    cout << a;
    cout << " b:";
    cout << b;
}
```

Изход

```
a:4 b:7
```

- Програмата отпечатва на екрана финалните стойности на *a* и *b* (съответно *4* и *7*).
- Обърнете внимание, че стойността на променливата *a* не е повлияна от последната промяна на *b*, въпреки че в по-горна команда сме декларирали, че *a = b*.

Оператор за присвояване (=)

- Операторите за присвояване сами по себе си се третират като изрази, които се оценяват.
  - *Присвояването има стойност, като за вградените типове тази стойност е стойността присвоена при операцията.*

```
y = 2 + (x = 5);
```

  - *В този израз, на* `y` *се присвоява резултатът от събирането между* `2` *и стойността на друг израз, съдържащ присвояване, който от своя страна има стойност* `5`*.*

- Горният пример е еквивалентен на:

```
x = 5;
y = 2 + x;
```

  - *Последното действие е да се присвои* `7` *на* `y`*.*

- В C++ е валиден и израз от вида на:

```
x = y = z = 5;
```

  - *В този израз се присвоява* `5` *на всички променливи:* `x`*,* `y` *и* `z`*; винаги от дясно на ляво.*

Аритметични операции (+, -, *, /, %)

- Аритметични операции, които се поддържат от C++:
  - *Операциите събиране, изваждане, умножение и деление съответстват точно на математическите операции.*
  - *Последната операция модул представена от знака за процент (%), връща остатъкът от деление на две стойности.*

```
x = 11 % 3;
```

  - *Променливата x съдържа стойност 2, тъй като при делене на 11 с 3 резултатът е 3 с остатък 2.*

Операция	Описание
+	Събиране
–	Изваждане
*	Умножение
/	Деление
%	Модул

Комбинирано присвояване

- Операциите за комбинирано присвояване (+=, -=, *=, /=, %=, >>=, <<=, &=, ^=, |=) променят текущата стойност на променливата като извършват операция над нея.
  - *Еквивалентни са на присъединяване на резултата от операция на първия операнд:*

Израз	Еквивалент
y += x;	y = y + x;
x -= 5;	x = x - 5;
x /= y;	x = x / y;
price *= units + 1;	price = price * (units+1);

Комбинирано присвояване

```
// оператори за комбинирано присвояване
#include <iostream>
using namespace std;

int main ()
{
    int a, b=3;
    a = b;

    a+=2;     // еквивалентно на израза a = a + 2

    cout << a;
}
```

Изход:

5

Инкремент и декремент (++, --)

- Някои изрази могат да се съкратят още повече: операторът за инкремент (++) и операторът за декремент (--) увеличава или намалява с единица стойността, която се съхранява в променливата.
  - *Тези операции са еквивалентни съответно на +=1 и на -=1.*

```
++x;
x+=1;
x=x+1;
```

  - *Следователно всички изрази от примера са еквивалентни един на друг по функционалност; и трите увеличават стойността на x с 1.*
  - *Съвременните компилатори автоматично оптимизират кода ни и трите израза водят до един и същ изпълним код.*

Инкремент и декремент (++, --)

- Особеност на този оператор е, че **може да се използва както като префиксен, така и като суфиксен**.
  - *Оператора може да бъде записан както пред името на променливата (**++x**), така и след нея (**x++**).*
- Въпреки че в прости изрази като ***x++*** или ***++x***, и двата имат едно и също значение; в други изрази, в които се оценява стойност на инкремет или декремент, може да имат много съществени разлики.
  - *В случай, че инкремента като операция се използва като префиксна(**++x**) пред стойността, изразът се оценява с крайната стойност на **x**, след като **x** вече е увеличена.*
  - *В случаите, когато операцията е използвана като суфиксна (постфиксна) **x++**, стойността отново се увеличава с **1**, но изразът се оценява със стойността, която **x** е имала преди да се увеличи.*

Инкремент и декремент (++, --)

Пример 1	Пример 2

x = 3;
y = ++x;
/* x съдържа 4, y съдържа 4 */
	x = 3;
	y = x++;
	/* x съдържа 4, y съдържа 3*/

- В Пример 1 стойността, която се свързва с *y* е стойността на *x* след като е увеличено.
- В Пример 2 стойността, която се свързва с *y* е стойността на *x* преди да е увеличено.

Релационни и операции за сравнение

- Два израза могат да се сравняват, чрез използването на релационни или оператори за сравнение.
  - *Например, когато ни се налага да проверяваме дали две стойности са равни или една е по-голяма от друга стойност.*
- Резултатът от такива операции е или истина или лъжа (напр., булева стойност).
- Релационните оператори в C++:

Оператор	Описание
==	Еквивалентно на
!=	Различно от
<	по-малко
>	по-голямо
<=	по-малко или равно
>=	по-голямо или равно

Релационни и операции за сравнение

- Пример:

```
(7 == 5)    // оценява се на false
(5 > 4)     // оценява се на true
(3 != 2)    // оценява се на true
(6 >= 6)    // оценява се на true
(5 < 5)     // оценява се на false
```

- Позволява се да се сравняват както числови константи, така и променливи.
- Нека предположим, че a = 2, b = 3 и c = 6, следователно:

```
(a == 5)    // оценява се на false, тъй като a не е равно на 5
(a*b >= c)  // оценява се на true, тъй като (2*3 >= 6) е истина
(b+4 > a*c) // оценява се на false, тъй като (3+4 > 2*6) не е истина
((b=2) == a) // оценява се на ???
```

Релационни и операции за сравнение

- ***!!! Операторът за присвояване (операторът с един знак за =) не е същият, като операторът за сравнение за равенство (еквивалентност оператор с два знака за равенство ==);***
- Операторът равно (=) присвоява стойността от дясно на променливата, която стои от ляво, докато операторът (==) сравнява дали стойностите от двете страни на оператора са еднакви.
  - *Следователно в последния израз(`(b=2) == a`), първо се присвоява стойността `2` на `b` и след това се сравнява с променливата `a` (която също съхранява стойност `2`), при което се получава `true`.*

Логически Операции (!, &&, ||)

- Операцията **!** в C++ съответства на логическата операция отрицание (**NOT**).
  - *Тя е едно-аргументна и има само един операнд, така че стои от ляво на аргумента си и обръща стойността му, т.е. резултатът от изпълнението е* **false** *, ако стойността на операнда е* **true**, *и* **true** *ако стойността на операнда е* **false**.

```
!(5 == 5)   // оценява се на false, тъй като изразът от дясно (5 == 5) е true
!(6 <= 4)   // оценява се на true, тъй като (6 <= 4) ще има стойност false
!true       // оценява се на false
!false      // оценява се на true
```

- Логическите операции **&&** и **||** се използват, тогава когато се оценяват няколко израза, за да се получи един релационен резултат.

Логически Операции (!, &&, ||)

- Операцията && съответства на булевата логическа операция **И**, която е истина, само тогава, когато и двата операнда са ***true***, и ***false*** във всички останали случаи.
  - *В таблицата по-долу е показан резултатът от операцията && при оценка на израза a && b:*

&& операция (И)		
a	b	a && b
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

- Операцията || съответства на булевата логическа операция **ИЛИ**, която е истина, ако кой да е операнд е ***true***, и е ***false*** само когато двата операнда са ***false***.
- В таблицата по-долу е показан резултатът от операцията || при оценка на израза a || b:

|| операция (ИЛИ)		
a	b	a || b
TRUE	TRUE	TRUE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

Логически Операции (!, &&, ||)

```
( (5 == 5) && (3 > 6) )  // оценява се на false ( true && false )
( (5 == 5) || (3 > 6) )  // оценява се true ( true || false )
```

- Когато използваме логически операции, C++ оценява само това, което е необходимо от ляво на дясно, за да стигне до комбиниран релационен резултат, като игнорира всичко останало.
  - *Поради тази причина в последния пример (`(5==5)||(3>6)`), C++ първо оценява стойността на израза `5==5` и ако е **`true`**, никога не проверява дали `3>6` е **`true`** или не.*
  - *Това е известно като оценка на късо-свързване.*

Оператор	късо-свързване
&&	Ако изразът от ляво е false, целият резултат е false (изразът от дясно никога не се оценява).
||	Ако изразът от ляво е true, целият резултат е true (изразът от дясно никога не се оценява).

Логически Операции (!, &&, ||)

- **Късо-свързване.** Особено важно е, когато изразът от дясно съдържа операция със странични ефекти, като промяна на стойности:

```
if ( (i < 10) && (++i < n) ) // i се инкрементира в условието
{
        /*...*/
}
```

  - *Комбинираният логически израз ще увеличи стойността на i с едно, но само ако условието от ляво на знака* **&&** *е* **true**, *в противен случай, условието от дясно* (**++i < n**) *никога не се оценява.*

Условен троен оператор (? :)

- Условният троен оператор оценява израз, като връща една стойност, ако този израз е истина (*true*) и друга стойност, ако този израз е лъжа (*false*) .
- Синтаксис:

условие ? резултат1 : резултат2

  - *Ако условието е **true**, целият израз се оценява със стойността на **резултат1**, в противен случай на **резултат2**.*

```
7 == 5 ? 4 : 3       // връща се стойност 3, тъй като 7 не е равно на 5.

7 == 5 + 2 ? 4 : 3   // връща се стойност 4, тъй като 7 е равно на 5+2.

5 > 3 ? a : b        // връща се стойността на a, тъй като 5 е по-голямо от 3.

a > b ? a : b        // оценява се на това, което е по-голямо, a или b.
```

Условен троен оператор (?)

- Пример:

```
// conditional operator
#include <iostream>
using namespace std;

int main ()
{
        int a,b,c;

        a=2;
        b=7;
        c = (a > b) ? a : b;

        cout << c << '\n';
   }
```

Изход:

```
7
```

- Променливата *a* има стойност *2*, а *b* стойност *7*, следователно при оценка на израза (*a > b*) се връща стойност ***false***, следователно първата стойност, която е специфицирана след въпросителната, се пренебрегва, за сметка на втората (изразът след двоеточието), която е *b* (със стойност *7*).

Оператор запетая (,)

- Операторът запетая (,) се използва, за да **раздели два или повече израза, които се използват там, където се очаква един единствен израз.**
  - *Когато се стигне до оценяване и изпълнение на изразите, под внимание се взима само най-десният израз.*

- Пример:

```
a = (b = 3, b + 2);
```

  - *В горния пример първо на* `b` *се присвоява стойност* `3`*, а след това изразът,* `b + 2` *се присвоява на променливата* `a`*. Така че, накрая променливата* `a` *ще съдържа стойност* `5`*, а променливата* `b` *стойност* `3`*.*

Побитови операции (&, |, ^, ~, <<, >>)

- Побитовите операции **модифицират променливите, използвайки битовете, които представят стойността в паметта**.

Оператор	Значение	Описание
&	AND	Побитово AND
|	OR	Побитово включващо OR
^	XOR	Побитово изключващо OR
~	NOT	Побитово логическо ДОПЪЛВАНЕ ДО 1
<<	SHL	Побитово отместване на ляво
>>	SHR	Побитово отместване на дясно

Преобразуване на типа

- Операторът за **явно преобразуване на типа** **позволяват, да се конвертира стойност от един тип в друг.**
  - *Най-простият начин, за да се постигне явно преобразуване на типа (наследява от езика C), е да се предшества израза, който трябва да бъде преобразуван от новия тип заграден в скоби (()):*

```
int i;
float f = 3.14;
 i = (int) f;
```

  - *В примера стойността 3.14 се преобразува към цяло число (3), остатъкът е загубен. Операторът за преобразуване е (int).*

- За да се постигне същият ефект в C++ се използва функционалната нотация, където типа предшества израза за преобразуване, който е заграден в скоби:

```
i = int (f);
```

- **Неявно преобразуване на типа - Изпълнява се автоматично, тогава когато в един израз участват стойности от различен тип.**
  - *Преобразуването става винаги към по-общия тип*
  - *Пример:*

*3.14 + 7 – 2 / 3 * 2.1*

sizeof

- Този оператор **приема един параметър, който може да е или име на тип или променлива**.
  - *Резултат от прилагането на оператора е размера на този тип, променлива или обект в байтове.*
  - *Пример:*

```
size_t x;
x = sizeof (char);
```

  - *В примера, x приема стойност 1, тъй като char е тип, който заема 1 байт в паметта.*
- Стойността, която се връща от оператора *sizeof* е константа известна по време на компилацията, така че тя се определя преди изпълнението на програмата.

Приоритет на операциите

- В един израз могат да бъдат използвани множество операции.
  - *Например:*

```
x = 3 + 9 % 2;
```

  - *В C++, горният пример винаги ще присвои стойност 4 на променливата x, защото приоритетът на операцията % е по-висок от приоритета на операцията +, следователно винаги се изпълнява преди него.*

- Части от изразите могат да се ограждат в скоби, за да се промени реда на изпълнение на операциите, или за по-голяма четливост на кода.
  - *Например:*

```
x = 5 + (7 % 2);    // x = 6
x = (5 + 7) % 2;    // x = 0
```

Приоритет на операциите

Ниво	Вид	Операция	Описание	Изпълнение
1	Област на видимост	::	Достъп до обхват на висимост	Ляво на дясно
2	Постфиксен (унарен)	++ --	Постфиксен инкремент / декремент	Ляво на дясно
		()	Параметри на функция	
		[]	subscript	
		. ->	Достъп до член	
3	Префиксен (унарен)	++ --	Префиксен инкремент / декремент	Дясно на ляво
		~ !	Побитово NOT / логическо NOT	
		+ -	Унарен префиксен знак	
		& *	Референция / дереференция	
		new, *delete*	Алокация / делокация	
		sizeof	Размер на параметъра	
		(тип)	C-стил на препобразуване на типа	
4	Указател към член	.* ->*	достъп до указател	Ляво на дясно
5	Скалиращи аритметични	* / %	Умножение, деление, модул	Ляво на дясно
6	Аритметични събиращи	+ -	Събиране, изваждане	Ляво на дясно
7	Побитово изместване	<< >>	Изместване на ляво и на дясно	Ляво на дясно

Приоритет на операциите

Ниво	Вид	Операция	Описание	Изпълнение
8	Сравнение	< > <= >=	Оператори за сравнение	Ляво на дясно
9	Еквивалентност	== !=	Еквивалентност, различие	Ляво на дясно
10	И	&	Побитово И	Ляво на дясно
11	Изключващо ИЛИ	^	Побитово изключващо ИЛИ	Ляво на дясно
12	Включващо ИЛИ	|	Побитово ИЛИ	Ляво на дясно
13	Конюнкция	&&	Логическо И	Ляво на дясно
14	Дизункция	||	Логическо ИЛИ	Ляво на дясно
15	Изрази на ниво присвояване	= *= /= %= += -= >>= <<= &= ^= |=	Комбинирано присвояване	Дясно на ляво
		? :	Условен оператор	
16	Последователности	,	Разделител запетая	Ляво на дясно

- **Когато в един израз има две операции с един и същи приоритет, изпълнението определя, коя ще се изпълни първа**: или от ляво надясно или от дясно наляво.
- Заграждането в скоби на всички подизрази (дори за тези, за които не е необходимо поради техния приоритет) повишава четливостта на кода.

Вход-изход в C++

- За да можем да се използват входно-изходните операции в езика C++, е необходимо да се включат следната библиотека:

```
#include <iostream>
```

- Включването на библиотеката, само по себе си не е достатъчно, за да се използват този входно-изходните операции, тъй като обектите, чрез които се извършва вход и изход са дефинирани в именуваното пространство ***std***:

```
using namespace std;
```

- За да може да се извърши операцията изход в конзолата, то трябва да се използва обекта ***cout***, заедно с операция за побитово изместване на ляво **<<**:

```
cout << "What number would you enter? ";
```

- За да може да се извършва операция вход от клавиатурата, то трябва да се използва обекта ***cin***, заедно с операция за побитово изместване на дясно **>>**:

```
int number;
cin >> number;
```

Размяна на стойности

- Често при решаване на различни задачи, се налага осъществяване на размяна на стойности.
  - ***Размяната на стойности може да се реализира или чрез трета променлива, или чрез аритметични и логически операции (ако няма достатъчно памет).***
- Размяна на стойности чрез аритметични операции:
  - *Чрез събиране и изваждане:*

```
a = a + b;
b = a - b;
a = a - b;
```

  - *Чрез умножение и деление:*

```
a = a * b;
b = a / b;
a = a / b;
```

- !!! ***Има и други варианти. Кои са те? Кой от горните два варианта е по-надежден и по-ефективен и защо?***

Побитови операции

- В паметта на компютъра всеки обект се разполага като двоично число (последователност от 0 и 1).
  - *Една машинна дума представлява определен брой битове, в зависимост от конкретната реализация на компютъра (8, 16, 32 битови компютри).*
- Побитовите операции интерпретират операнда или операндите си като съвкупност от битове.
  - *Позволяват на програмиста да провери или присвои стойности на отделни битове или множество битове.*
- Побитовите операции се означават със следните символи: &, |, ^, ~, >>, <<.

Побитови операции

- & (побитово логическо И):

  1 & 1 = 1

  1 & 0 = 0

  0 & 1 = 0

  0 & 0 = 0

  - *Пример:*

  1 1 0 1 0
  &
  1 0 0 0 0
  ----
  1 0 0 0 0

- *Използва се обикновено за нулиране на отделни битове в числената стойност на резултата.*
- *Ако някой бит в единия или другия операнд има нулева стойност, съответния по позиция бит в резултата също ще има нулева стойност.*

Побитови операции

- | (побитово логическо ИЛИ):

  1 | 1 = 1

  1 | 0 = 1

  0 | 1 = 1

  0 | 0 = 0

  - *Пример:*

    1 1 0 1 0 1
    |
    0 1 0 0 1 0
    ----
    1 1 0 1 1 1

- *Операцията може да се разглежда като обратна на побитовото И, следователно отделни битове в резултата, могат да се направят единици.*

Побитови операции

- ^ (побитово логическо ИЗКЛЮЧВАЩО ИЛИ (XOR))

  1 ^ 1 = 0

  1 ^ 0 = 1

  0 ^ 1 = 1

  0 ^ 0 = 0

  - *Пример:*

    1 0 0 1 0

    ^

    1 1 0 0 1

    ----

    0 1 0 1 1

  - *Определени битове в резултата се установяват като единица само когато съответните битове в операндите са различни.*

Побитови операции

- ~ (побитово логическо ДОПЪЛВАНЕ ДО 1)
  - *Едноместна операция с асоциативно правило на изпълнение отдясно наляво.*
  - *Операнд може да бъде всеки валиден израз, имащ резултат цяла стойност.*
  - *При изпълнението на операцията всеки бит на операнда се променя с обратната си стойност, т.е. 0 става 1, а 1 става 0.*
  - *От това веднага следва, че двукратното изпълнение на операцията възстановява числото в първоначален вид.*
  - *Пример:*

```
~11010101
 ————————
 00101010
```

Побитови операции

- >> (побитово отместване на дясно)
  - *Двуоперандна операция. Първият операнд е валиден израз връщащ число, а вторият брой битове, с които отместваме вдясно (т.е. винаги е цяло число).*
  - *Пример:*

    01111000 >> 3

    00001111
  - *Ефектът от примера е, че все едно три пъти сме разделили числото на две. Например* 120 >> 3 *ще върне резултат* 15 ((120 / 2) / 2) / 2.

Побитови операции

- << (побитово отместване на ляво)
  - *Обратната операция на изместването на дясно.*
  - *Пример:*

    00001111 << 3

    01111000
  - *Ефектът от операцията е равносилен на умножение на числото три пъти по две. Например* 10 << 3 *ще върне резултат* 80 (10 * 2 * 2 * 2)*.*

Побитови операции

- При операциите изместване вляво или вдясно трябва да знаете, че изместените битове се губят.
  - *Например:*

    00001111 >> 1

    00000111 << 1

    00001110
  - *Затова резултатът от операциите е неопределен, ако броя битове е по-голям или равен на дължината на първия операнд.*

Примерен код

```
#include <iostream>
using namespace std;

int main(){

    unsigned char b1 = 0003;
    //осмична бройна система, 3 в десетична бройна система или 0011 в двоична

    unsigned char b2 = 0007;
    //7 в десетична бройна система или 0111 в двоична

    //побитово И
    cout << "b1&b2 = " << (b1&b2) << endl;
    //0011&0111=0011 -> 3 в десетична

    //побитово ИЛИ
    cout << "b1|b2 = " << (b1|b2) << endl;
    //0011&0111=0111 -> 7 в десетична


    //побитово отместване на дясно
    cout << (120>>3) << endl; //равносилно на (((120/2)/2)/2) = 15

    //побитово отместване на ляво
    cout << (20<<4) << endl; //равносилно на (((20*2)*2)*2)*2 = 320

    return 0;
}
```

Домашна работа

Задача 1. Съставете C++ конзолна програма, чрез която да се изисква от потребителя да въведе стойности за променливи от всички вградени в езика примитивни типове данни (int, char, bool, ...).

Задача 2. Съставете C++ конзолна програма, чрез която да се извършат и отпечатат на екрана всички стойности на математически изрази, чрез стойности на две целочислени променливи, въведени от потребителя. Използвайте +, -, *, /, %.

Задача 3. Съставете C++ конзолна програма, чрез която да напишете изрази, използващи съкратените математически операции, върху променливи със стойности въведени от потребителя. След всяка промяна да се отпечатва стойността на променливите.

Задача 4. Съставете C++ конзолна програма, чрез която да се разменят стойности на две променливи, чрез побитови операции.

Задача 5. Да се проучи, за какво служат побитовите операции и кога са приложими.

Домашна работа

Задача 6. Да се напише програма, която намира и извежда координатите на медицентъра на триъгълник, зададен чрез координати на трите му върха.

Задача 7. Да се напише програма, която намира разстоянията от дадена точка $T(x, y)$ до страните на квадрата с върхове:

а) (0 , 0), (1 , 0), (0 , 1) и (1 , 1);

б) (1 , 1), (-1 , 1), (-1 , - 1) и (-1 , 1).

Задача 8. Да се напише програма, която намира разстоянията от дадена точка $T(x, y)$ до страните на триъгълника с върхове:

а) (0 , 0), (1 , 0) и (0 , 1);

б) (0 , 0), (1 , 0) и (1 , 1);

в) (-1 , 0), (0 , 0) и (1 , 1).

Задача 9. Даден е триъгълник със стойностите на ъглите си и радиуса на описаната окръжност. Да се напише програма, която намира лицето на триъгълника.

Домашна работа

Задача 10. 600 грама 56%-на азотна киселина са смесени с 800 грама 35%-на азотна киселина. Да се напише програма, която намира процентното съдържание на получената смес.

Задача 11. Да се напише програма, която по дадени ***i***, ***j***, ***x***, ***y*** определя дали на ***i***-та позиция на ***x*** и на ***j***-та позиция в ***y*** има еднакви битове (номерацията на битовете започва от 0). Работете с булеви изрази (оператори за сравнение, логически оператори, побитови оператори)! Не използвайте условни оператори (***if***, ***if-else***, ***switch***)!

Задача 12. Да се напише програма, която разменя ***i***-ти и ***j***-ти бит на число без знак.

Задача 13. Да се напише програма, която по зададени стойности на реалните променливи ***a***, ***b*** и ***c*** намира:

а) min{a+b+c, a.b.c} + 1.5;

б) max{a2 - b3 + c, a - 4.5 b, 1.5 a + 3.5 b - 8 c} - 7.5.

Домашна работа

Задача 14. Да се напише програма, която увеличава по-малкото от две дадени цели неравни числа 3 пъти, а по-голямото число намалява 5 пъти.

Задача 15. Дадени са три числа ***a***, ***b*** и ***c***. Да се напише програма, в резултат от изпълнението на която, ако е в сила релацията $a \leq b \leq c$, числата се заменят с техните абсолютни стойности, в противен случай числата се удвояват. Работете с булеви изрази (оператори за сравнение, логически оператори, побитови оператори)! Не използвайте условни оператори (***if***, ***if-else***, ***switch***)!

Благодаря за вниманието !!!

Въпроси и коментари